Installation

This is the authoritative installation and data-download guide for the artifact. Hardware and software prerequisites are listed in `REQUIREMENTS.md`; experiment commands are in `README.md`.

The complete training and evaluation workflow uses Linux ARM64 (`aarch64`). The separate Intel/x86-64 environment in Section 7 is used **only** to reproduce the Intel measurements in Figure 4.

1. Choose a workspace

The repository, dependencies, dataset, and generated maps require substantial storage. Choose any filesystem with at least 65 GB free; it does not need to be your home directory. Use an absolute path without a trailing slash:

```
export ARTIFACT_ROOT="/absolute/path/to/emsoft-artifact"

export REPO_ROOT="${ARTIFACT_ROOT}/GRB-Search-Pipeline"
export COSIPY_ROOT="${ARTIFACT_ROOT}/cosipy"
export DATA_ROOT="${ARTIFACT_ROOT}/transients"
export DEPS_ROOT="${ARTIFACT_ROOT}/dependencies"

mkdir -p "${ARTIFACT_ROOT}" "${DEPS_ROOT}"
```

The commands in this document and in `README.md` use these variables. Re-export them when starting a new shell.

2. Obtain the source code

Clone the artifact repository and the external COSIpy dependency as sibling directories:

```
cd "${ARTIFACT_ROOT}"

git clone \
    https://github.com/Daisy0419/GRB-Search-Pipeline.git \
    "${REPO_ROOT}"

git clone \
    --branch tsmap-artifact \
    --single-branch \
    https://github.com/McKelvey-Engineering-CSE/cosipy.git \
    "${COSIPY_ROOT}"
```

The exact artifact snapshot is also archived at Zenodo. If that archive is used instead of GitHub, extract it under `ARTIFACT_ROOT` and set `REPO_ROOT` to the actual extracted directory. COSIpy must still be obtained separately as shown above.

The initial artifact checkout does not contain a `cosipy/` subdirectory. `COSIPY_ROOT` refers to the external repository cloned by this step.

3. ARM64 Python environment for the complete workflow

This section configures the environment used for:

- visualizing the distributed results;
- running the full training and evaluation workflow; and
- running the three Jetson measurements in Figure 4.

Verify the architecture:

```
uname -m
```

The expected output is:

```
aarch64
```

3.1 Install Miniconda

Skip the installer if a suitable Conda installation is already available. Otherwise, install the ARM64 version under the chosen workspace:

```
export CONDA_DIR="${DEPS_ROOT}/miniconda-arm64"
export MINICONDA_INSTALLER="${ARTIFACT_ROOT}/miniconda-arm64.sh"

wget -q \
    https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-aarch64.sh
\
    -O "${MINICONDA_INSTALLER}"

bash "${MINICONDA_INSTALLER}" -b -p "${CONDA_DIR}"
rm "${MINICONDA_INSTALLER}"
```

Add Miniconda to `PATH` and initialize it in the current shell:

```
export PATH="${CONDA_DIR}/bin:${PATH}"
. "${CONDA_DIR}/etc/profile.d/conda.sh"
conda config --set channel_priority flexible
```

These commands must be repeated in a new shell unless the corresponding `PATH` setting and Conda initialization are added to the shell startup file. Alternatively, run `conda init bash` once and open a new shell.

3.2 Create the ARM64 environment

```
cd "${REPO_ROOT}"
conda env create -f cosipy-312-deps.yaml
conda activate cosipy-312
```

Install the external COSIpy checkout and Jupyter Notebook:

```
cd "${COSIPY_ROOT}"
git switch tsmap-artifact

python -m pip install "poetry-core>=2,<3"
python -m pip install --no-deps -e .
python -m pip install notebook
```

Verify the Python installation:

```
python -c \
    "import cosipy; import hdf5plugin; print('Python environment: OK')"
```

4. Native dependencies for the complete workflow

The C++ GCP programs require CMake, LEMON, HDF5, and HighFive. These dependencies are not required merely to visualize the distributed results or to run the optional Figure 4 mapping benchmark.

4.1 Check the CMake version

Building the current HDF5 source requires **CMake 3.26 or later**:

```
cmake --version
```

If the first line reports a version older than 3.26, install a newer CMake before continuing. One user-local option, after activating the Conda environment, is:

```
python -m pip install "cmake>=3.26"
hash -r
cmake --version
```

4.2 Install LEMON 1.3.1

```
cd "${DEPS_ROOT}"

wget -c http://lemon.cs.elte.hu/pub/sources/lemon-1.3.1.tar.gz
tar -xzf lemon-1.3.1.tar.gz

export LEMON_SOURCE_DIR="${DEPS_ROOT}/lemon-1.3.1"
export LEMON_BUILD_DIR="${LEMON_SOURCE_DIR}/build"

cmake -S "${LEMON_SOURCE_DIR}" -B "${LEMON_BUILD_DIR}"
cmake --build "${LEMON_BUILD_DIR}" -j
```

4.3 Install HDF5

Install HDF5 under `DEPS_ROOT`; administrator privileges are not required:

```
export HDF5_SOURCE_DIR="${DEPS_ROOT}/hdf5-source"
export HDF5_BUILD_DIR="${DEPS_ROOT}/hdf5-build"
export HDF5_ROOT="${DEPS_ROOT}/hdf5"

git clone --depth 1 \
    https://github.com/HDFGroup/hdf5.git \
    "${HDF5_SOURCE_DIR}"

cmake -S "${HDF5_SOURCE_DIR}" -B "${HDF5_BUILD_DIR}" \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_INSTALL_PREFIX="${HDF5_ROOT}" \
    -DCMAKE_INSTALL_LIBDIR=lib \
    -DBUILD_SHARED_LIBS=ON \
    -DBUILD_TESTING=OFF \
    -DHDF5_BUILD_EXAMPLES=OFF \
    -DHDF5_BUILD_CPP_LIB=ON \
    -DHDF5_BUILD_HL_LIB=ON

cmake --build "${HDF5_BUILD_DIR}" -j
cmake --install "${HDF5_BUILD_DIR}"
```

Configure and verify HDF5:

```
export LD_LIBRARY_PATH="${HDF5_ROOT}/lib:${LD_LIBRARY_PATH:-}"

test -f "${HDF5_ROOT}/include/hdf5.h"
ls "${HDF5_ROOT}"/lib/libhdf5*
```

4.4 Install HighFive

HighFive is header-only and does not require a separate build:

```
export HIGHFIVE_ROOT="${DEPS_ROOT}/HighFive"

git clone --depth 1 --recursive \
    https://github.com/highfive-devs/highfive.git \
    "${HIGHFIVE_ROOT}"

test -f "${HIGHFIVE_ROOT}/include/highfive/H5File.hpp"
```

4.5 Build the C++ executables

```
export HDF5_ROOT="${DEPS_ROOT}/hdf5"
export HIGHFIVE_ROOT="${DEPS_ROOT}/HighFive"
export LEMON_SOURCE_DIR="${DEPS_ROOT}/lemon-1.3.1"
export LEMON_BUILD_DIR="${LEMON_SOURCE_DIR}/build"
export LD_LIBRARY_PATH="${HDF5_ROOT}/lib:${LD_LIBRARY_PATH:-}"

cmake \
    -S "${REPO_ROOT}/search-planning" \
    -B "${REPO_ROOT}/search-planning/build"

cmake --build "${REPO_ROOT}/search-planning/build" -j
```

Verify the two executables:

```
test -x "${REPO_ROOT}/search-planning/build/sp_train"
test -x "${REPO_ROOT}/search-planning/build/sp_verify"
echo "C++ executables: OK"
```

`sp_train` generates GCP training outcomes, while `sp_verify` evaluates the generated test maps.

5. Download the experiment data

The detector models, simulated transient datasets, and Figure 4 benchmark inputs are distributed separately through Zenodo:

- Record: https://zenodo.org/records/21497891
- Archive: `transients.tar.gz`
- Download size: approximately 16.2 GB
- MD5: `ba6f9511b99b5fb3faaebfaae01e03ac`

Download the archive into the selected workspace:

```
export
TRANSIENTS_URL="https://zenodo.org/records/21497891/files/transients.tar.gz
?download=1"
export TRANSIENTS_ARCHIVE="${ARTIFACT_ROOT}/transients.tar.gz"
```

```
curl \
    --location \
    --fail \
    --continue-at - \
    --output "${TRANSIENTS_ARCHIVE}" \
    "${TRANSIENTS_URL}"
```

Alternatively, use:

```
wget -c "${TRANSIENTS_URL}" -O "${TRANSIENTS_ARCHIVE}"
```

Verify the Zenodo MD5 checksum:

```
echo "ba6f9511b99b5fb3faaebfaae01e03ac  ${TRANSIENTS_ARCHIVE}" \
    | md5sum --check
```

Extract the archive. It contains a top-level `transients/` directory:

```
tar -xzf "${TRANSIENTS_ARCHIVE}" -C "${ARTIFACT_ROOT}"
```

Verify the required inputs:

```
test -f "${DATA_ROOT}/models/adapt_response_w_area.h5"
test -f "${DATA_ROOT}/models/adapt_bkg_model.h5"
test -d "${DATA_ROOT}/emsoft_training_short"
test -d "${DATA_ROOT}/emsoft_training_longlow"
test -d "${DATA_ROOT}/emsoft_test_short"
test -d "${DATA_ROOT}/emsoft_test_longlow"
test -d "${DATA_ROOT}/dc3_benchmark_data"
echo "Experiment data: OK"
```

After successful extraction and verification, the compressed archive may be removed to recover space:

```
rm "${TRANSIENTS_ARCHIVE}"
```

6. Final smoke test and next steps

Restore the ARM64 environment and dependency paths if necessary:

```
```

```
export CONDA_DIR="${DEPS_ROOT}/miniconda-arm64"
export PATH="${CONDA_DIR}/bin:${PATH}"
. "${CONDA_DIR}/etc/profile.d/conda.sh"
conda activate cosipy-312

export HDF5_ROOT="${DEPS_ROOT}/hdf5"
export LD_LIBRARY_PATH="${HDF5_ROOT}/lib:${LD_LIBRARY_PATH:-}"
```

Run the checks:

```
python -c \
    "import cosipy; import hdf5plugin; print('Python environment: OK')"

test -x "${REPO_ROOT}/search-planning/build/sp_train" \
    && test -x "${REPO_ROOT}/search-planning/build/sp_verify" \
    && echo "C++ executables: OK"

test -f "${DATA_ROOT}/models/adapt_response_w_area.h5" \
    && test -f "${DATA_ROOT}/models/adapt_bkg_model.h5" \
    && echo "Data and models: OK"
```

Next:

- follow README Section 2 to visualize the distributed results;
- follow README Section 3 to regenerate the complete experiments; or
- follow README Section 4 to run the optional Figure 4 benchmark.

7. Optional Intel x86-64 environment for Figure 4

> **Scope:** This environment is used only for the five Intel measurements in Figure 4. It is not used for the full training or policy-evaluation workflow. The Jetson measurements in Figure 4 use the ARM64 environment from Section 3.

Perform this setup on a Linux x86-64 machine. First verify:

```
uname -m
```

The expected output is:

```
x86_64
```

Use the same workspace layout from Section 1. If the artifact repository, COSIpy checkout, and data archive are not already present on this machine, complete Sections 1, 2, and 5 first.

Install x86-64 Miniconda under the selected workspace, unless a suitable Conda installation is already available:

```
export CONDA_DIR_INTEL="${DEPS_ROOT}/miniconda-x86_64"
export MINICONDA_INSTALLER="${ARTIFACT_ROOT}/miniconda-x86_64.sh"

wget -q \
    https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh \
    -O "${MINICONDA_INSTALLER}"

bash "${MINICONDA_INSTALLER}" -b -p "${CONDA_DIR_INTEL}"
rm "${MINICONDA_INSTALLER}"
```

Add this installation to `PATH` and initialize Conda in the current shell:

```
export PATH="${CONDA_DIR_INTEL}/bin:${PATH}"
. "${CONDA_DIR_INTEL}/etc/profile.d/conda.sh"
conda config --set channel_priority flexible
```

Create the Figure-4-only Intel environment:

```
cd "${REPO_ROOT}"
conda env create -f cosipy-312-intel.yml
conda activate cosipy-312-intel
```

Install the external COSIpy checkout and Jupyter Notebook:

```
cd "${COSIPY_ROOT}"
git switch tsmap-artifact

python -m pip install "poetry-core>=2,<3"
python -m pip install --no-deps -e .
python -m pip install notebook
```

No LEMON, user-built HDF5, HighFive, or C++ GCP build is required for this optional mapping-only benchmark. Continue with README Section 4.1.